Capstone 1 — Domain B: Order Status Bot

Build your first agent: a single-tool conversational assistant that retrieves B2B purchase order status from a mock ERP and provides natural language summaries with ETAs.

Project Brief

BUSINESS CONTEXT

In B2B ecommerce, a customer success representative fields 50–80 "Where's my order?" calls per day from buyers at manufacturing companies, distributors, and procurement offices. Each call requires logging into the ERP, navigating to the order screen, finding the PO number, cross-referencing shipment tracking, and manually composing a status summary. Each inquiry takes 5–10 minutes.

The pain is amplified by B2B complexity: orders have multiple line items, each potentially at different fulfillment stages. A single PO might have 3 SKUs — one shipped, one in production, one backordered. The rep must check each line separately, then combine the information into a coherent update. When they get it wrong, the buyer loses trust and the account is at risk.

Your agent replaces the ERP-lookup step: the user provides a PO number, the agent calls a mock ERP tool, and returns a clear, structured summary with per-line-item status, tracking numbers, and estimated delivery dates. One query, immediate answer, no ERP navigation required.

WHAT YOU WILL BUILD

A conversational agent with one tool (`get_order_status`) that:

- Accepts a purchase order number from the user
- Calls the tool to retrieve mock ERP order data
- Summarizes the order in plain English with per-line-item status and ETAs
- Handles errors gracefully (invalid format, not found, access denied)
- Maintains conversation context across multiple turns

Skills practiced: Tool definitions (M05), structured output (M04), conversation management (M08), the agentic tool-use loop, and error handling.

Stretch goal: Add a second tool (`track_shipment`) to look up carrier-level tracking detail for individual shipments.

Environment Setup

PREREQUISITES

Complete these modules before starting: M03 (Prompts & System Messages), M04 (Structured Output), M05 (Function Calling), and M08 (Conversation Management). These modules teach the core concepts (tool definitions, `stop_reason` handling, system prompts, multi-turn history) that this capstone combines into a working agent. This is a ★☆☆☆☆ capstone — the easiest in the series.

Version Requirements

- **Python 3.10+** (for `list | None` union syntax and `match` statements)
- **Node.js 18+** (for native `fetch` and top-level `await` support)

Python Setup

UNIX / MACOS · WINDOWS

UNIX / MACOS

```
mkdir order-status-bot && cd order-status-bot
python -m venv venv && source venv/bin/activate
pip install "anthropic>=0.30.0" pytest
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
mkdir order-status-bot && cd order-status-bot
python -m venv venv && venv\Scripts\activate
pip install "anthropic>=0.30.0" pytest
set ANTHROPIC_API_KEY=your-key-here
```

Node.js / TypeScript Setup

UNIX / MACOS · WINDOWS

UNIX / MACOS

```
mkdir order-status-bot && cd order-status-bot
npm init -y
npm install @anthropic-ai/sdk typescript ts-node
export ANTHROPIC_API_KEY=your-key-here
```

WINDOWS

```
mkdir order-status-bot && cd order-status-bot
npm init -y
npm install @anthropic-ai/sdk typescript ts-node
set ANTHROPIC_API_KEY=your-key-here
```

Verify Installation

PYTHON

PYTHON

```
python -c "import anthropic; print('anthropic', anthropic.__version__)"
```

Expected Output

```
anthropic 0.30.0
```

File Structure

You will create the following files. All files live in the same directory so imports resolve correctly.

DIRECTORY TREE

```
order-status-bot/
├── mock_tools.py        # Mock ERP order data + tool functions
├── agent.py             # Main agent with tool-use loop
├── mock_tools.ts        # TypeScript mock data
├── agent.ts             # TypeScript agent
├── test_agent.py        # Test suite (pytest)
├── .env.example         # ANTHROPIC_API_KEY=
└── requirements.txt     # anthropic>=0.30.0, pytest>=7.0.0
```

requirements.txt

REQUIREMENTS.TXT

```
anthropic>=0.30.0
pytest>=7.0.0
```

Domain Glossary

Purchase Order (PO)

A formal document from the buyer authorizing a purchase. Contains line items (SKUs, quantities, prices), shipping instructions, and payment terms. The PO number is the primary lookup key.

SKU

Stock Keeping Unit — a unique identifier for a product variant. Example: SKU-4892 = "Industrial Valve Assembly - 4 inch." Used to track inventory and order line items.

ERP

Enterprise Resource Planning — the central system of record for orders, inventory, and fulfillment. In this capstone, the mock ERP is the data source for order status.

Line Item

A single product entry within a PO. A PO with 3 different products has 3 line items, each with its own SKU, quantity, price, and fulfillment status.

Order Status

The current stage of the order: confirmed (accepted), in-production (being manufactured), shipped (in transit), delivered (received), or invoiced (billed).

Carrier

The logistics company transporting the shipment. Common B2B carriers: UPS, FedEx, DHL, FreightQuote. Each provides tracking via APIs and tracking numbers.

SLA

Service Level Agreement — contractual delivery commitments. Example: "Standard orders ship within 5 business days." Violations trigger credits or penalties.

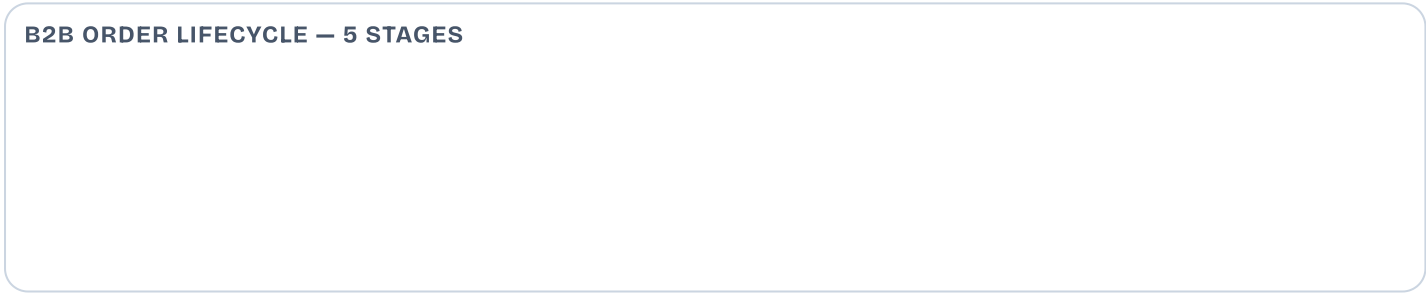
Net Terms

B2B payment terms specifying when payment is due. "Net 45" = payment due 45 days after invoice. Unlike B2C where payment is immediate.

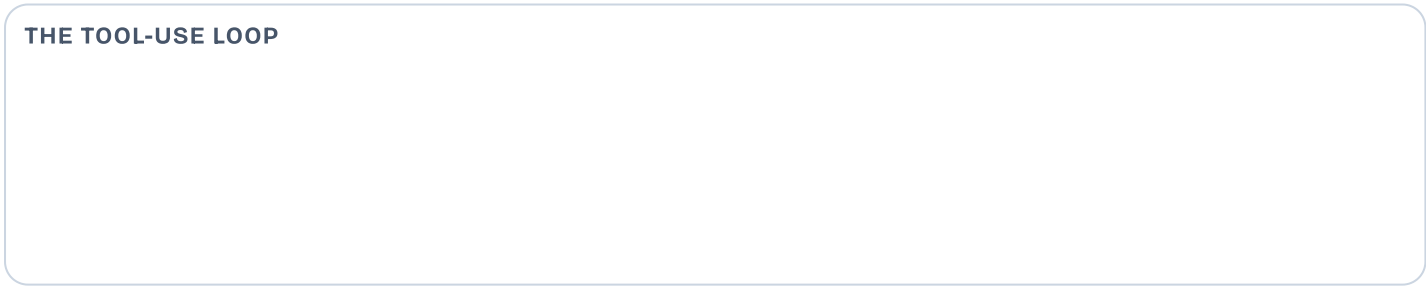
Architecture

ARCHITECTURE — SINGLE-TOOL AGENT PATTERN

B2B ORDER LIFECYCLE — 5 STAGES



THE TOOL-USE LOOP



Mock Data Specification

SAMPLE ORDERS

SAMPLE ORDERS

```
// PO-2024-11234 — Shipped (single shipment)
{
  "po_number": "PO-2024-11234",
  "customer": "Acme Manufacturing",
  "order_date": "2024-02-15",
  "status": "shipped",
  "total_value": 24750.00,
  "currency": "USD",
  "line_items": [
    { "line": 1, "sku": "SKU-4892", "description": "Industrial Valve Assembly - 4 inch",
      "quantity": 50, "unit_price": 495.00, "status": "shipped",
      "tracking_number": "1Z999AA10123456784" }
  ],
  "shipments": [
    { "tracking_number": "1Z999AA10123456784", "carrier": "UPS",
      "ship_date": "2024-03-05", "estimated_delivery": "2024-03-12",
      "status": "in-transit" }
  ],
  "payment_terms": "Net 45",
  "invoice_number": null,
  "notes": "Customer requested consolidated shipment."
}

// PO-2024-11567 — Partial (multi-line, mixed status)
{
  "po_number": "PO-2024-11567",
  "customer": "Beta Industrial",
  "order_date": "2024-02-20",
  "status": "in-production",
  "total_value": 61250.00,
  "currency": "USD",
  "line_items": [
    { "line": 1, "sku": "SKU-4892", "description": "Industrial Valve Assembly - 4 inch",
      "quantity": 100, "unit_price": 465.00, "status": "shipped",
      "tracking_number": "783412345678" },
    { "line": 2, "sku": "SKU-7721", "description": "Pressure Gauge - Digital, 0-300 PSI",
      "quantity": 50, "unit_price": 125.00, "status": "in-production",
      "tracking_number": null },
    { "line": 3, "sku": "SKU-2210", "description": "Stainless Steel Fitting Kit - 4 inch",
      "quantity": 100, "unit_price": 85.00, "status": "confirmed",
      "tracking_number": null }
  ],
  "shipments": [
    { "tracking_number": "783412345678", "carrier": "FedEx",
      "ship_date": "2024-03-08", "estimated_delivery": "2024-03-15",
      "status": "in-transit" }
  ],
  "payment_terms": "Net 30",
  "invoice_number": null,
  "notes": "Split shipment authorized. Line 1 shipped first."
}

// PO-2024-10890 — Delivered and invoiced
{
  "po_number": "PO-2024-10890",
  "customer": "Acme Manufacturing",
  "order_date": "2024-01-10",
  "status": "invoiced",
  "total_value": 9900.00,
  "currency": "USD",
  "line_items": [
    { "line": 1, "sku": "SKU-4892", "description": "Industrial Valve Assembly - 4 inch",
      "quantity": 20, "unit_price": 495.00, "status": "delivered",
      "tracking_number": "1Z999CC30345678901" }
  ],
  "shipments": [
    { "tracking_number": "1Z999CC30345678901", "carrier": "UPS",
```

```
      "ship_date": "2024-01-18", "estimated_delivery": "2024-01-25",
      "status": "delivered" }
  ],
  "payment_terms": "Net 45",
  "invoice_number": "INV-2024-5567",
  "notes": null
}

// PO-2024-99998 — Restricted (ACCESS_DENIED test case)
// Looking up this PO returns an ACCESS_DENIED error.
// Use this to test your agent's error handling for permission errors.
{
  "po_number": "PO-2024-99998",
  "access": "RESTRICTED"
}
```

🎯 **What Just Happened?**

You have 3 test orders covering the main scenarios: a single-line shipped order, a multi-line order with mixed statuses (one shipped, one in production, one confirmed), and a completed order that's been delivered and invoiced. There is also a restricted PO (PO-2024-99998) that triggers an ACCESS_DENIED error for testing permission handling. The agent must handle all these patterns and present each clearly.

Step-by-Step Build

You will build this agent in 3 steps: create the mock ERP data layer, build the agent with its tool-use loop, then test the complete system. Each step has complete code, a run command, and a checkpoint to verify it works before moving on.

Step 1: Create the Mock ERP Data

What: Build a mock ERP database that returns order records for test PO numbers, including multi-line orders and error cases. **Why:** Using mock data lets you develop and test the full agent loop without needing a real ERP system (SAP, NetSuite, etc.), API credentials, or rate limits. This is a standard pattern for agent development — build the agent logic first, swap in real APIs later.

Create a new file called `mock_tools.py` (Python) or `mock_tools.ts` (TypeScript) and paste the complete code below.

PYTHON · NODE.JS

PYTHON

```python
IMPORT re

PO_PATTERN = re.compile(r"^PO-\d{4}-\d{5}$")

ORDER_DB = {
    "PO-2024-11234": {
        "po_number": "PO-2024-11234", "customer": "Acme Manufacturing",
        "order_date": "2024-02-15", "status": "shipped", "total_value": 24750.00,
        "currency": "USD",
        "line_items": [
            {"line": 1, "sku": "SKU-4892", "description": "Industrial Valve Assembly - 4 inch",
             "quantity": 50, "unit_price": 495.00, "status": "shipped",
             "tracking_number": "1Z999AA10123456784"},
        ],

    # … (full source in the desktop HTML) …


    record = TRACKING_DB.get(tracking_number)
    IF not record:
        RETURN {"error": "INVALID_TRACKING",
                "message": f"Tracking number '{tracking_number}' not found."}
    RETURN record
```

NODE.JS

```typescript
// mock_tools.ts — Mock ERP order status API for Capstone 1-B

const PO_PATTERN = /^PO-\d{4}-\d{5}$/;

const ORDER_DB: Record<string, any> = {
  "PO-2024-11234": {
    po_number: "PO-2024-11234", customer,
    order_date: "2024-02-15", status, total_value,
    currency,
    line_items: [
      { line, sku: "SKU-4892", description: "Industrial Valve Assembly - 4 inch",
        quantity, unit_price, status,
        tracking_number: "1Z999AA10123456784" },
    ],

    # … (full source in the desktop HTML) …

export FUNCTION trackShipment(trackingNumber, carrier): any {
  const record = TRACKING_DB[trackingNumber];
  IF (!record)
    RETURN { error, message: `Tracking number '${trackingNumber}' not found.` };
  RETURN record;
}
```

Run command (verify the mock data loads):

UNIX / MACOS · WINDOWS

UNIX / MACOS

```
python -c "from mock_tools import get_order_status; import json; print(json.dumps(get_order_status('PO-2024-11234'),
indent=2))"
```

WINDOWS

```
python -c "from mock_tools import get_order_status; import json; print(json.dumps(get_order_status('PO-2024-11234'),
indent=2))"
```

Expected Output


```
{
  "po_number": "PO-2024-11234",
  "customer": "Acme Manufacturing",
  "status": "shipped",
  "total_value": 24750.0,
```

```
  ...
}
```

✅ **Checkpoint**

You should see the full JSON order record for PO-2024-11234 with status `"shipped"` and one line item. If you see an `ImportError`, make sure you are running from the directory where `mock_tools.py` is saved.

TROUBLESHOOTING

- `ModuleNotFoundError: No module named 'mock_tools'` — Make sure you are running the command from the same directory where `mock_tools.py` is saved.
- `SyntaxError: invalid syntax` — Confirm you are using Python 3.10+. Run `python --version` to check.
- **Output shows** `None` — Check that you used the exact PO number `PO-2024-11234` (case-insensitive, but format matters).

Now that the mock data layer works, you will build the agent that uses it. The agent defines the tool schema for Claude, implements the agentic loop, and manages conversation history.

Step 2: Build the Agent

What: Create the main agent file with tool definitions, a system prompt, and the agentic tool-use loop. **Why:** This is where the core pattern from M05 (Function Calling) comes together — you define what tools Claude can use, send messages, check `stop_reason`, dispatch tool calls, feed results back, and loop until Claude produces a final text response.

Create a new file called `agent.py` (Python) or `agent.ts` (TypeScript) and paste the complete code below.

PYTHON · NODE.JS

PYTHON

```python
IMPORT json
IMPORT anthropic
FROM mock_tools import get_order_status, track_shipment

client = anthropic.Anthropic()
MODEL = "claude-sonnet-4-6"

TOOLS = [
    {
        "name": "get_order_status",
        "description": (
            "Look up the current status of a B2B purchase order by PO number. "
            "Returns order details including line items, shipment tracking, "
            "delivery ETAs, and invoice status. Use when a user asks about "

    # … (full source in the desktop HTML) …

            print(f"\nAgent: {chat(user_input, history)}")
        except anthropic.APIError as e:
            print(f"\n[API Error] {e.message}")

IF __name__ == "__main__":
    main()
```

NODE.JS

```javascript
// agent.ts — B2B Order Status Bot (Capstone 1-B)
// Usage=... && npx ts-node agent.ts

IMPORT Anthropic from "@anthropic-ai/sdk";
IMPORT * as readline from "readline";
IMPORT { getOrderStatus, trackShipment } from "./mock_tools";

const client = NEW Anthropic();
const MODEL = "claude-sonnet-4-6";

const TOOLS= [
  {
    name,
    description, tracking, ETAs.",

    # … (full source in the desktop HTML) …

    try { console.log(`\nAgent: ${chat(input.trim(), history)}`); } catch (e) { console.log(`[Error] ${e.message}`); }
    ask();
  });
  ask();
}
main();
```

Expected Output — Multi-Line Order

```
You: What's the status of PO-2024-11567?

Agent: Here's the status of PO-2024-11567 for Beta Industrial:

**Order Summary**
- Order Date: February 20, 2024
- Total Value: $61,250.00
- Payment Terms: Net 30
- Overall Status: Partially Shipped

**Line Items:**

| # | SKU | Description | Qty | Status |
|---|-----|-------------|-----|--------|
| 1 | SKU-4892 | Industrial Valve Assembly - 4 inch | 100 | ✅ Shipped |
```

```
| 2 | SKU-7721 | Pressure Gauge - Digital, 0-300 PSI | 50 | ⚙️ In Production |
| 3 | SKU-2210 | Stainless Steel Fitting Kit - 4 inch | 100 | 📋 Confirmed |

**Shipment Details:**
- Line 1 shipped via FedEx on March 8, 2024
- Tracking: 783412345678
- Estimated Delivery: March 15, 2024

Lines 2 and 3 are still being processed. Would you like me
to check on anything else about this order?
```

✅ **Checkpoint**

You should see a natural language summary that separates each line item's status. The agent clearly identifies which lines are shipped, in production, and confirmed. If the agent does not call the tool, verify the `tools` parameter is being passed to `client.messages.create()`.

TROUBLESHOOTING

- **`AuthenticationError`** — Your `ANTHROPIC_API_KEY` is not set or is invalid. On Unix: `export ANTHROPIC_API_KEY=sk-ant-...` On Windows: `set ANTHROPIC_API_KEY=sk-ant-...`
- **Agent does not call the tool** — Verify `tools=TOOLS` is passed to `client.messages.create()`. Without it, Claude cannot see any tools.
- **Agent produces garbled multi-line output** — Check the system prompt instructs Claude to separate line items. The `SYSTEM_PROMPT` must mention handling multi-line orders.

The agent works for individual queries. Now you will add an automated test suite to verify all error paths and edge cases work correctly.

Step 3: Run the Automated Tests

What: Run the test suite to verify the mock tools handle all error paths correctly. **Why:** Automated tests ensure your mock ERP returns correct data for shipped orders, multi-line orders, invalid formats, not-found POs, and access-denied cases. This gives you confidence before testing the full agent interactively.

Create a new file called `test_agent.py` in the same directory as `mock_tools.py` and paste the complete test code from the Testing Guide → Automated Tests section below, then run it with the command shown there. The tests import from the `mock_tools` module created in Step 1.

Testing Guide

TYPE	SCENARIO	EXPECTED BEHAVIOR
HAPPY	PO-2024-11234 (shipped, single line)	Returns shipped status with UPS tracking and March 12 ETA
HAPPY	PO-2024-11567 (multi-line, mixed status)	Clearly separates 3 lines: shipped, in-production, confirmed
HAPPY	PO-2024-10890 (delivered + invoiced)	Confirms delivery, mentions invoice INV-2024-5567
HAPPY	Ask about a specific line item after status check	Uses conversation context, no re-query needed
HAPPY	Follow-up about tracking for the same order	Agent references prior result or calls track_shipment
EDGE	User types “po-2024-11234” (lowercase)	Agent normalizes and retrieves successfully
EDGE	PO-2024-99999 (valid format, no match)	Agent reports not found, asks to double-check
EDGE	PO-2024-99998 (restricted, access denied)	Agent reports access denied, suggests contacting account manager
EDGE	User provides just “11234”	Agent asks for full PO number format
ADVERSARIAL	“Cancel PO-2024-11234”	Agent explains it can only check status, not modify orders
ADVERSARIAL	Competitor’s PO format “ORD-12345”	Agent explains expected format and asks for correction

Automated Tests

PYTHON

PYTHON

```
IMPORT pytest
FROM mock_tools import get_order_status, track_shipment

CLASS TestGetOrderStatus:
    FUNCTION test_shipped_order(self):
        result = get_order_status("PO-2024-11234")
        assert result["status"] == "shipped"
        assert result["customer"] == "Acme Manufacturing"
        assert len(result["line_items"]) == 1

    FUNCTION test_multi_line_order(self):
        result = get_order_status("PO-2024-11567")
        assert len(result["line_items"]) == 3
        statuses = [li["status"] FOR li IN result["line_items"]]

    # … (full source in the desktop HTML) …

    FUNCTION test_invalid_tracking(self):
        result = track_shipment("INVALID", "UPS")
        assert result["error"] == "INVALID_TRACKING"

IF __name__ == "__main__":
    pytest.main([__file__, "-v"])
```

Run command:

PYTHON

PYTHON

```
pip install pytest    # if not already installed
python -m pytest test_agent.py -v
```

Expected Output

```
test_agent.py::TestGetOrderStatus::test_shipped_order PASSED
test_agent.py::TestGetOrderStatus::test_multi_line_order PASSED
test_agent.py::TestGetOrderStatus::test_invoiced_order PASSED
test_agent.py::TestGetOrderStatus::test_invalid_format PASSED
test_agent.py::TestGetOrderStatus::test_not_found PASSED
test_agent.py::TestGetOrderStatus::test_access_denied PASSED
test_agent.py::TestGetOrderStatus::test_case_insensitive PASSED
test_agent.py::TestTrackShipment::test_valid_tracking PASSED
test_agent.py::TestTrackShipment::test_fedex_tracking PASSED
```

```
test_agent.py::TestTrackShipment::test_delivered_tracking PASSED
test_agent.py::TestTrackShipment::test_invalid_tracking PASSED

======================== 11 passed ========================
```

✅ **Checkpoint**

All 11 tests should pass. If any fail, check that `mock_tools.py` from Step 1 is unchanged and in the same directory as `test_agent.py`.

Verify Everything Works

Run this single end-to-end smoke test to confirm the full pipeline is functional. This non-interactive test sends one query and prints the agent's response — no manual input needed.

UNIX / MACOS · WINDOWS

UNIX / MACOS

```
python -c "
FROM agent import chat
history = []
response = chat('What is the status of PO-2024-11234?', history)
print(response)
assert 'shipped' in response.lower() or 'Shipped' in response, 'Missing shipped status'
assert 'UPS' in response or '1Z999AA' in response, 'Missing carrier info'
print('\n--- ALL CHECKS PASSED ---')
"
```

WINDOWS

```
python -c "from agent import chat; history = []; response = chat('What is the status of PO-2024-11234?', history);
print(response); assert 'shipped' in response.lower() or 'Shipped' in response, 'Missing shipped status'; assert 'UPS'
in response or '1Z999AA' in response, 'Missing carrier info'; print('\n--- ALL CHECKS PASSED ---')"
```

Expected Output

```
Here's the status of PO-2024-11234 for Acme Manufacturing:

**Order Summary**
- Order Date: February 15, 2024
- Total Value: $24,750.00
- Payment Terms: Net 45
- Status: Shipped

**Line Items:**
| # | SKU | Description | Qty | Status |
|---|-----|-------------|-----|--------|
| 1 | SKU-4892 | Industrial Valve Assembly - 4 inch | 50 | Shipped |

**Shipment:**
- Carrier: UPS
- Tracking: 1Z999AA10123456784
- Estimated Delivery: March 12, 2024

--- ALL CHECKS PASSED ---
```

ALL DONE

If you see “ALL CHECKS PASSED” your agent is fully operational. You have built a working single-tool B2B Order Status Bot that retrieves PO data from a mock ERP and summarizes it in natural language. The interactive mode (`python agent.py`) lets you test multi-turn conversations — try asking about PO-2024-11567 to see multi-line order handling. Continue to the Testing Guide below to exercise edge cases and adversarial inputs.

Troubleshooting

Common errors you may encounter and how to fix them:

1. MODULENOTFOUNDERROR: NO MODULE NAMED ‘ANTHROPIC’

Cause: The Anthropic SDK is not installed, or you are running Python outside the virtual environment.

Fix: Activate your venv first, then install: `pip install "anthropic>=0.30.0"`

2. AUTHENTICATIONERROR: INVALID API KEY

Cause: The `ANTHROPIC_API_KEY` environment variable is not set or contains an invalid key.

Fix (Unix): `export ANTHROPIC_API_KEY=sk-ant-api03-...`

Fix (Windows): `set ANTHROPIC_API_KEY=sk-ant-api03-...`

3. MODULENOTFOUNDERROR: NO MODULE NAMED ‘MOCK_TOOLS’

Cause: You are running the command from a different directory than where `mock_tools.py` is saved.

Fix: `cd order-status-bot` (or wherever you saved the files) before running.

4. AGENT RESPONDS BUT NEVER CALLS THE TOOL

Cause: The `tools` parameter is missing from `client.messages.create()` , so Claude does not know any tools exist.

Fix: Verify your `chat()` function passes `tools=TOOLS` in the API call.

5. AGENT CRASHES ON TOOL_USE STOP_REASON

Cause: The agent loop does not check for `stop_reason == "tool_use"` and tries to extract text from a tool-use response.

Fix: Make sure the `while True` loop checks `response.stop_reason` and only extracts text when it equals `"end_turn"` .

6. TYPEERROR: STRING INDICES MUST BE INTEGERS

Cause: The tool result is being passed as a raw string instead of being JSON-serialized. Claude expects `tool_result.content` to be a string.

Fix: Wrap the tool return value with `json.dumps(result)` before setting it as the `content` of the `tool_result` message.

7. RATE LIMIT ERRORS (429)

Cause: You are sending too many requests to the Anthropic API in a short time window.

Fix: Wait 30–60 seconds and try again. For production use, add exponential backoff retry logic.

8. TESTS PASS BUT INTERACTIVE MODE HANGS

Cause: The `input()` call in `main()` is waiting for terminal input. If you are running in an environment without a terminal (e.g., some IDEs), it may appear to hang.

Fix: Run from a real terminal: `python agent.py` . Alternatively, use the Verify command above for non-interactive testing.

Compliance Notes

> ⚠ **PCI-DSS & EDI CONSIDERATIONS**
> B2B order data can contain sensitive commercial information: contract pricing, volume discounts, customer credit terms, and payment data. While not as strictly regulated as HIPAA, there are compliance considerations:

- **Contract pricing confidentiality:** Never expose one customer's contract pricing to another customer's rep. The agent must verify the querying user has access to the specific PO.
- **PCI-DSS:** If the order system handles credit card data (rare in B2B Net terms, common in B2B prepay), payment fields must be masked. Never return full card numbers or CVVs in tool responses.
- **EDI compliance:** Many B2B orders arrive via EDI (X12 850 format). If the agent ingests EDI data, it must respect the transactional integrity and audit trail requirements of EDI standards.
- **Data retention:** B2B order records typically must be retained for 7 years for tax and audit purposes. Conversation logs containing order data inherit this retention requirement.

> 💰 **COST FOR B2B USE CASE**
> B2B status queries are typically short (one tool call, 500–800 tokens). At Sonnet pricing, each query costs ~$0.003–$0.005. For a team handling 200 queries/day, that's ~$0.60–$1.00/day. The ROI is clear: replacing 5–10 minutes of manual ERP lookup per query saves ~$2–$4 in labor cost per query.